

MCP y sus bases en la ingeniería de prompt

Montivero Miotto, Tito Santino

Laboratorio de Investigación de Software

Departamento de Ingeniería en Sistemas de Información

Facultad Regional Córdoba - Universidad Tecnológica Nacional.

Abstract

Los avances recientes en el campo de la inteligencia artificial (AI) y particularmente en los modelos de lenguaje de gran escala (LLMs) han generado un ecosistema tecnológico en constante expansión. En esta rama de la (AI) podemos encontrar desarrollos recientes respecto a su arquitectura, a sus aplicaciones, a sus optimizaciones. En este contexto, el Model Context Protocol (MCP) se presenta como un estándar emergente que busca facilitar la integración entre LLMs y aplicaciones tradicionales, ofreciendo un marco seguro y estructurado para el intercambio de contexto. Este trabajo analiza en detalle el funcionamiento del MCP, identificando las técnicas de ingeniería de prompts y de gestión dinámica del contexto que incorpora, así como sus implicaciones para la optimización del rendimiento de los modelos. Adicionalmente, se discuten los beneficios y limitaciones del protocolo frente a otros enfoques de enriquecimiento contextual, con el objetivo de contribuir a una mejor comprensión de su papel en el diseño de sistemas inteligentes basados en NLP (natural language processing).

necesidad de establecer mecanismos estandarizados que permitan su integración con aplicaciones de software tradicionales. En este marco surge el Model Context

Palabras Clave

Model Context Protocol, LLMs, MCP, NLP, ingeniería de prompts, optimización de contexto, procesamiento de lenguaje natural.

Introducción

El acelerado progreso en el desarrollo de modelos de lenguaje de gran escala (LLMs, por sus siglas en inglés)[1] ha impulsado la

Protocol¹, cuyo objetivo es proporcionar un lenguaje común de interacción entre modelos y herramientas externas, ampliando así el espectro de tareas que los sistemas de procesamiento de lenguaje natural (PLN) pueden abordar. El protocolo se estructura en dos componentes fundamentales:

- **Core:** define la información que el modelo requiere, el formato en el que debe recibirla y los parámetros necesarios para ejecutar las aplicaciones asociadas.
- **Normalizaciones:** establecen aspectos esenciales para la comunicación confiable, incluyendo la seguridad, el protocolo de transporte y la arquitectura de referencia.

Actualmente, el MCP ofrece dos especificaciones principales de transporte: una basada en Standard IO y otra en Server-Sent Events (SSE)², diseñadas para garantizar la interoperabilidad dentro de una arquitectura cliente-servidor. Esta división permite que el protocolo sea suficientemente flexible para adaptarse a diferentes entornos de ejecución, manteniendo al mismo tiempo un marco de comunicación seguro y consistente.

El objetivo de este trabajo es realizar un análisis crítico del MCP, examinando su funcionamiento interno y los principios conceptuales que sustentan su diseño, con el propósito de construir una guía que oriente la toma de decisiones y establezca buenas prácticas para futuras implementaciones. De esta manera, se busca no solo comprender los fundamentos técnicos del protocolo, sino también evaluar su aplicabilidad y su potencial para optimizar el intercambio contextual en sistemas inteligentes basados en LLMs

Ingeniería de prompt

En múltiples escenarios, una consulta simple no resulta suficiente para que un modelo de lenguaje de gran escala (LLM) genere una respuesta satisfactoria. La denominada ingeniería de prompts (Prompt Engineering) [2] se ha consolidado como un conjunto de técnicas clave para ampliar las capacidades de los modelos sin necesidad de modificar sus parámetros internos. Al enriquecer el contexto de entrada, es posible mejorar significativamente la calidad de las respuestas, reduciendo la dependencia de enfoques más costosos como el fine-tuning especializado en tareas únicas[3].

Este principio no es exclusivo del MCP; se relaciona estrechamente con la estrategia de Retrieval-Augmented Generation (RAG) [4], en la cual el modelo accede a información adicional para complementar su conocimiento y producir salidas más precisas.

El MCP surge con el objetivo de estandarizar este proceso de enriquecimiento contextual, proporcionando, en palabras de sus desarrolladores, “una manera segura, estandarizada y sencilla para dar a los sistemas de IA el contexto que necesitan”. Sin embargo, un análisis más detallado revela que el protocolo no solo adopta estos conceptos, sino que también integra diversas técnicas de la ingeniería de prompts como mecanismos fundamentales para su funcionamiento.

Funcionamiento

El flujo de trabajo del Model Context Protocol (MCP) se basa en la interacción entre un cliente, responsable de gestionar las consultas hacia el modelo de lenguaje, y un servidor, encargado de generar el contexto adicional requerido.

La secuencia típica de operación puede describirse de la siguiente manera:

1. El cliente envía una petición inicial al LLM.
2. El LLM, al procesar la consulta, determina la necesidad de invocar

¹ <https://modelcontextprotocol.io>

² <https://html.spec.whatwg.org/multipage/server-sent-events.html>

- una herramienta externa mediante el servidor MCP y devuelve una instrucción estándar de tipo call/tool.
3. El cliente redirige la solicitud hacia el servidor MCP.
 4. El servidor procesa la petición y responde con el contexto o los resultados correspondientes.
 5. El cliente reinyecta dicha información en el LLM para completar la tarea.
 6. Finalmente, el cliente entrega la respuesta final al usuario.

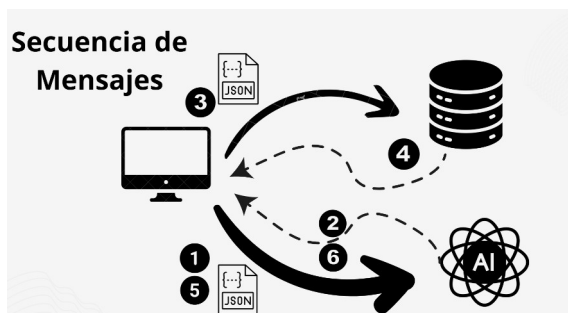


Figura 1. Flujo de interacción cliente-servidor en el MCP.

Además de este ciclo principal, el protocolo contempla comunicaciones auxiliares esenciales para su correcto funcionamiento como:

Inicialización: intercambio de capacidades y parámetros entre cliente y servidor.

Discovery: mecanismo que permite al cliente obtener dinámicamente la lista de comportamientos disponibles mediante el comando tools/list.

Notificaciones unidireccionales: mensajes emitidos por el servidor para indicar cambios, que requieren un nuevo proceso de discovery por parte del cliente.

La fase de discovery implementa principios de la ingeniería de prompts Few-Shot[5], ya que el servidor puede proporcionar ejemplos

de uso de sus herramientas, permitiendo que el LLM adquiera contexto explícito sobre cómo invocarlas y qué resultados esperar.

Asimismo, el diseño del MCP está orientado a habilitar la resolución de tareas complejas mediante cadenas de acciones. Al ofrecer parámetros, descripciones y resultados estandarizados para cada herramienta, el protocolo configura una arquitectura modular de bloques lógicos que el LLM puede combinar (ver fig. 2). Esta estructura favorece la aplicación de técnicas como Chain-of-Thought (CoT) [6] y Logic-of-Thought (LoT) [7] sobre la entrada, incrementando la capacidad del modelo para alcanzar objetivos mediante secuencias lógicas de pasos.

Ejemplo ilustrativo:

Considérese el caso de un usuario que consulta a un LLM: “¿Cuál fue el promedio de ventas mensuales de la base de datos de clientes durante el último trimestre?”. El modelo, al no disponer de esa información internamente, tendría que generar una respuesta basada únicamente en el texto de la consulta y en su conocimiento general. Por otro lado, si cuenta con una base de datos con las ventas del último trimestre en un servidor MCP, puede responder con una instrucción para acceder a la herramienta de consulta SQL expuesta. El cliente redirige la solicitud al servidor, el cual ejecuta la query en la base de datos y devuelve el resultado estructurado en formato JSON {“promedio”: 3520}. El cliente reinyecta este contexto en el LLM, que finalmente

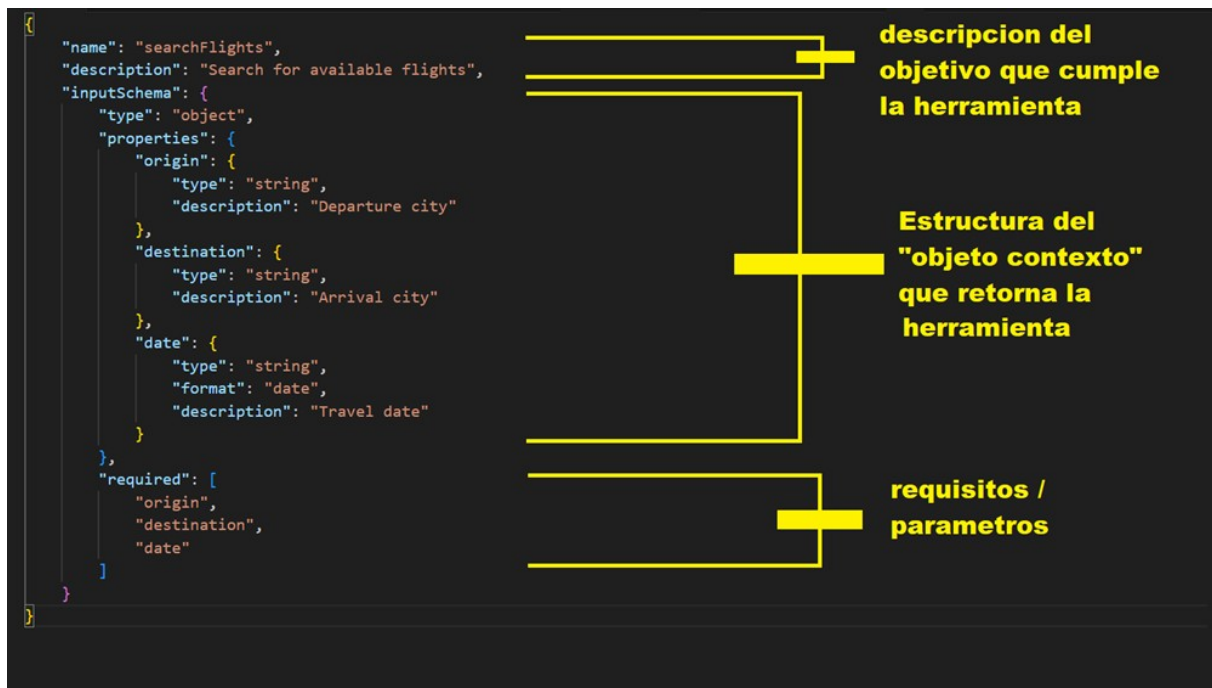


Figura 2. estructura de la respuesta “Discovery” de un servidor MCP (se ocultaron otros campos, para demostrar la estructura de bloque de acción)

responde al usuario: “El promedio de ventas mensuales en el último trimestre fue de 3.520 unidades.”.

Este ejemplo demuestra cómo el MCP permite extender de manera flexible las capacidades de un LLM, combinando su poder “inferencial” con el acceso dinámico a fuentes externas de información.

Experimento

En el laboratorio realizamos algunos experimentos con esta tecnología. El primero consistió en obtener información actualizada en tiempo real a partir de páginas web, de manera que un modelo de lenguaje de gran escala pudiera utilizar dichos datos como contexto en consultas sobre eventos recientes. Para este propósito planteamos desarrollar un servidor MCP desarrollado en java con el framework spring. El cual contenía tres herramientas distintas.

FetchInfoHTML: que retorna el archivo HTML de una página web.

CaptureWeb: que toma un screenshot de una página web y lo guarda en un repositorio local.

ReadFile: que expone el contenido de un archivo desde un repositorio local.

Como componente cliente utilizamos el software de Anthropic, Claude desktop³ y como modelo utilizamos la api de Claude Sonnet 4.

El propósito de esta configuración fue examinar los desafíos emergentes en escenarios de uso real del MCP, más allá de sus especificaciones teóricas. El primer obstáculo significativo surgió durante la fase de inicialización del servidor. Los mensajes de inicialización generados por el servidor no fueron interpretados correctamente por el cliente Claude, lo que resultó en fallos de conexión recurrentes. Este comportamiento puso de manifiesto la sensibilidad del protocolo a implementaciones específicas y la necesidad de un manejo robusto de los estados iniciales de la comunicación.

Una vez solventados los problemas de inicialización, se observó que el modelo Claude Sonnet 4 demostró un desempeño consistente en la selección e invocación de las herramientas disponibles. En todos los casos de prueba, el LLM identificó correctamente la herramienta pertinente para

³ <https://claude.ai/download>

cada consulta y generó las instrucciones de ejecución adecuadas.

No obstante, se identificó una limitación operacional crítica relacionada con la política de tokens máxima impuesta por Anthropic para sus modelos. Un caso ilustrativo se presentó en el experimento en el cual se intentó inyectar texto derivado de una imagen como contexto. Se instruyó al agente para que generara una captura de una página web mediante la herramienta CaptureWeb y, posteriormente, utilizara ReadFile para interpretarla con el fin de integrarla luego como contexto en una consulta subsiguiente. Este procedimiento buscaba simular un flujo de trabajo en el que los recursos visuales obtenidos del entorno digital se transformaran en insumos procesables por el modelo y aunque los LLMs modernos pueden codificar/procesar imágenes en representaciones binarias, el número de tokens requerido para procesar una imagen completa resultó prohibitivo. La solución adoptada fue utilizar un sistema OCR (Optical character recognition) para preprocesar la imagen y convertirla en texto conciso, lo que incrementó de manera significativa la eficiencia del flujo y la pertinencia del contexto. Esta restricción imposibilita la ejecución de experimentos más avanzados que involucraran cadenas largas de acciones o el procesamiento de contextos sustanciales, tales como la inyección de múltiples capturas de pantalla o largos documentos HTML. Como consecuencia, fue necesario reevaluar el stack tecnológico utilizado y considerar alternativas que permitieran una mayor flexibilidad en el manejo del volumen contextual para experimentos futuros.

El protocolo y sus limitaciones

Aunque el Model Context Protocol (MCP) representa un avance significativo al integrar de manera nativa prácticas de gestión de contexto, su adopción plantea diversos desafíos que requieren un análisis previo.

En primer lugar, la eficacia del protocolo depende estrechamente del modelo de lenguaje con el que se integre. Los ejemplos y descripciones que el servidor MCP ofrece para guiar al LLM pueden resultar más o menos útiles según el entrenamiento y las capacidades intrínsecas del modelo. Esta dependencia reproduce una de las principales limitaciones de la ingeniería de prompts: su carácter estocástico y poco predecible obliga a un proceso iterativo de prueba y error para alcanzar configuraciones óptimas, tal como se ha discutido en trabajos sobre Prompt Design and Engineering [8].

En segundo lugar, el protocolo establece un estándar de intercambio basado en JSON, pero no impone restricciones sobre la estructura o calidad de los datos internos. Esto traslada al desarrollador la responsabilidad de garantizar que la información proporcionada al modelo sea coherente, relevante y eficiente en términos de consumo de tokens.

En tercer lugar, si bien el protocolo ofrece la posibilidad de seleccionar entre distintos formatos de transporte, para implementaciones en entornos de producción, independientemente de la arquitectura de componentes, resulta altamente recomendable emplear el modo basado en Server-Sent Events (SSE). Aunque la especificación sustentada en Standard Input/Output (STDIO) es más sencilla de implementar, esta acapara por completo la salida serializada de los procesos en ejecución y dado que numerosas bibliotecas utilizan dicha salida para reportar advertencias, configuraciones u otros mensajes de estado, su uso puede generar conflictos de compatibilidad con el protocolo, comprometiendo la estabilidad del sistema.

Finalmente, la selección del modelo de lenguaje es un factor crítico. A medida que las tareas demandan cadenas de acciones más largas y complejas, se requiere un mayor poder de inferencia por parte del LLM. En ciertos escenarios, puede ser indispensable recurrir a modelos que implementen tecnologías de razonamiento más avanzadas, como el Logic-of-Thought (LoT), para mantener un desempeño adecuado.

En síntesis, si bien el MCP ofrece una base sólida para estandarizar la integración entre LLMs y herramientas externas, su efectividad está condicionada por factores como la calidad del modelo, el diseño de los datos de entrada y la correcta elección de técnicas de preprocesamiento. Estos elementos deben ser considerados cuidadosamente para evitar que el protocolo se convierta en una solución incompleta.

Investigaciones futuras

Como línea de trabajo futura, se plantea el desarrollo de un cliente propio que permita un control más detallado del flujo de interacción entre el modelo y el servidor MCP. Esta herramienta buscará no solo optimizar la gestión de las solicitudes y respuestas, sino también facilitar la recopilación de métricas precisas sobre el desempeño del protocolo en distintos escenarios de uso. Dichas métricas resultan fundamentales para evaluar la eficiencia del flujo de comunicación, la latencia entre eventos y la calidad del contexto inyectado, aspectos que hoy constituyen limitaciones observadas en implementaciones basadas exclusivamente en clientes genéricos.

Este avance permitira profundizar en el estudio de los componentes esenciales del protocolo, identificando cuáles de sus estrategias contribuyen de manera más significativa a la mejora del rendimiento general. Se proyecta avanzar hacia una

evaluación comparativa entre diferentes arquitecturas de implementación, tanto en entornos controlados como en casos de uso reales. Este enfoque permitirá sentar las bases para una futura versión del protocolo.

Conclusión

El MCP constituye un aporte relevante en el emergente campo de la gestión y optimización del contexto aplicado a modelos de lenguaje. Al estandarizar la interacción entre LLMs y aplicaciones externas, el protocolo facilita la construcción de sistemas más robustos y flexibles, capaces de abordar tareas que trascienden las limitaciones de los enfoques tradicionales.

No obstante, su adopción exige un conocimiento mínimo de los fundamentos que dieron origen a la ingeniería de prompts y las técnicas de enriquecimiento contextual. El desconocimiento de estas puede derivar en implementaciones ineficientes o incluso ineficaces, desaprovechando el potencial del protocolo.

En consecuencia, el MCP debe ser entendido no como una solución definitiva, sino como un marco que amplía las posibilidades de integración de los LLMs. Su efectividad dependerá, en última instancia, de la capacidad de investigadores y desarrolladores para seleccionar modelos adecuados, diseñar contextos de calidad y aplicar estrategias de prompting que maximicen el rendimiento del ecosistema.

Agradecimientos

Al equipo del laboratorio de investigación por sus consejos y aportes. Y en particular al profesor Nicolas Hernandez por su guía y apoyo en dicha investigación.

Referencias

- [1] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," arXiv preprint arXiv:2308.10620, 2023.
- [2] Sahoo, P., Singh, A. K., Saha, S., Jain, V., Mondal, S., & Chadha, A. (2024). A Systematic Survey of Prompt Engineering in Large Language

Models: Techniques and Applications. arXiv (Cornell University).
<https://doi.org/10.48550/arxiv.2402.07927>

[3] Mou, L., Meng, Z., Yan, R., Li, G., Xu, Y., Zhang, L., & Jin, Z. (2016). How Transferable are Neural Networks in NLP Applications? arXiv (Cornell University).
<https://doi.org/10.48550/arxiv.1603.06111>

[4] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In Advances in Neural Information Processing Systems, 33, 9459–9474. Curran Associates, Inc.

[5] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., . . . Amodei, D. (2020). Language Models are Few-Shot Learners. arXiv (Cornell University).
<https://doi.org/10.48550/arxiv.2005.14165>

[6] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv (Cornell University).
<https://doi.org/10.48550/arxiv.2201.11903>

[7] Liu, T., Xu, W., Huang, W., Wang, X., Wang, J., Yang, H., & Li, J. (2024). Logic-of-Thought: Injecting Logic into Contexts for Full Reasoning in Large Language Models. arXiv (Cornell University).
<https://doi.org/10.48550/arxiv.2409.17539>

[8] Amatriain, X. (2024). Prompt Design and Engineering: Introduction and Advanced Methods. arXiv (Cornell University).
<https://doi.org/10.48550/arxiv.2401.14423>

Datos de Contacto:

*Tito Santino Montivero Miotto. CP: 5001.
zsantinoz@gmail.com.*